

PropertyPilot — Deployment Guide

Last updated: 2026-06-04 **Audience:** Anyone deploying or maintaining PropertyPilot — local Docker stack today, Render in production.

This document covers the containerization story end-to-end. It supplements [maintenance-guide.md](#), which covers the un-containerized local workflow.

1. Overview

PropertyPilot ships as three containers on a shared Docker network:

Container	Image	Purpose	Host port
propertypilot-postgres	postgres:16-alpine	Application database	5432
propertypilot-backend	propertypilot-backend (built from backend/Dockerfile)	Express API. Runs prisma migrate deploy on startup, then node dist/app.js.	4000
propertypilot-frontend	propertypilot-frontend (built from frontend/Dockerfile)	nginx serving the static bundle and reverse-proxying /api/* to the backend.	8080 → 80

All three are wired together by [docker-compose.yml](#) and an internal bridge network named `propertypilot`. The browser only ever talks to `localhost:8080` (the frontend); the API call goes via nginx's reverse proxy into the backend service, which talks to Postgres over the internal network.

2. Docker Files at a Glance

/	
├─ docker-compose.yml	# 3 services + 1 network + 1 named volume
├─ .dockerignore	# Root build-context excludes (used by both image builds)
├─ backend/	
│ └─ Dockerfile	# Multi-stage backend build
│ └─ .dockerignore	# Mirrors root rules for package-scoped builds
└─ frontend/	
└─ Dockerfile	# Multi-stage frontend build
└─ nginx.conf	# SPA fallback + /api/* reverse-proxy to `backend:4000`
└─ .dockerignore	# Mirrors root rules for package-scoped builds

Both Dockerfiles use the **repository root** as the build context. This is because npm workspaces share a single `package-lock.json` at the root, and a self-contained build can only stay reproducible if it sees that lockfile.

3. Backend Image

`backend/Dockerfile` is a two-stage build:

1. **builder** (node:20-alpine)

- Installs OS tooling (`python3 make g++ openssl libc6-compat`) for the `bcrypt` native build and the Prisma query engine.
- Copies just the package manifests, then runs `npm ci` . This keeps the install layer cached across source-only changes.
- Copies the backend source, runs `npm run prisma:generate -w backend` , and `npm run build -w backend` .

2. **runtime** (node:20-alpine)

- Installs only the runtime OS packages (`openssl libc6-compat dumb-init`).
- Copies the resolved `node_modules` from the builder. Two copies are required:
 - `/repo/node_modules/` → `/app/node_modules/` (hoisted deps: Express, Prisma client, bcrypt, etc.)
 - `/repo/backend/node_modules/` → `/app/backend/node_modules/` (npm workspaces leaves a few packages including the **prisma CLI** in the per-workspace folder rather than hoisting them; missing this copy is why the first build failed with `sh: ./node_modules/.bin/prisma: not found`).
- Copies `backend/dist` , `backend/prisma` , and the workspace package manifests.
- Switches to the non-root `node` user.
- **Entry command:** `dumb-init sh -c './node_modules/.bin/prisma migrate deploy && node ./dist/app.js'` . `prisma migrate deploy` is idempotent — it applies any pending migrations and is safe to run on every container start. `dumb-init` makes sure SIGTERM reaches Node so `docker compose down` terminates cleanly.

3.1 Why not `npx prisma migrate deploy`

Using `npx` resolves the bin from the current directory's `node_modules/.bin` . With workspaces, the prisma CLI is in `/app/backend/node_modules/.bin/prisma` , but the workspace root's bin folder doesn't include it. `npx` would fall back to downloading the latest Prisma from the registry — Prisma 7.x at the time of writing, which removed the `url = env("DATABASE_URL")` syntax our schema uses. Invoking the local binary directly avoids that.

4. Frontend Image

`frontend/Dockerfile` is also a two-stage build:

1. **builder** (node:20-alpine) — installs deps and runs `npm run build -w frontend` , producing `frontend/dist` .
2. **runtime** (nginx:1.27-alpine) — copies `frontend/dist` to `/usr/share/nginx/html` and replaces the default site config with [frontend/nginx.conf](#) .

The nginx config does three things:

- Serves the static bundle, with cache-busting headers on `index.html` and `Cache-Control: immutable` on `/assets/*` (which Vite fingerprints).
 - Provides the SPA fallback so React Router's client-side routes (`/properties` , `/leases` , etc.) all return `index.html` .
 - Reverse-proxies `/api/*` to `http://backend:4000/api/*` . The hostname `backend` is the compose service name on the shared internal network.
-

5. docker-compose.yml

Three services, one internal network, one named volume.

5.1 postgres

The pre-existing local Postgres container. Healthchecked with `pg_isready`. Data persists in the `protypilot_pgdata` named volume so `docker compose down` doesn't lose data.

5.2 backend

- `build.context: . + build.dockerfile: backend/Dockerfile`.
- Reads `JWT_SECRET` from the root `.env` (or the host environment). The compose file uses the `${JWT_SECRET:?...}` form, which **errors loudly if the value is missing** instead of silently falling back to a dev default.
- Overrides `DATABASE_URL` to point at the internal `postgres` service (not the host's `localhost:5432`).
- Sets `FRONTEND_URL=http://localhost:8080` so the CORS middleware allows requests coming from the frontend container (the browser's origin, since the user hits the frontend's port directly).
- `depends_on: postgres: condition: service_healthy` — the backend does not start until the database healthcheck passes.
- Healthcheck: `wget -qO- http://127.0.0.1:4000/api/health`. (Using `127.0.0.1` rather than `localhost` because the Alpine image's name resolver does not always map `localhost` cleanly when `wget` is invoked from the shell.)

5.3 frontend

- `build.context: . + build.dockerfile: frontend/Dockerfile`.
- Maps host port `8080` to nginx's internal `80`.
- `depends_on: backend: condition: service_started` — nginx will get `502` errors if it accepts requests before the backend is up.
- Healthcheck: `wget -qO- http://127.0.0.1/`.

5.4 Network and volumes

- Internal bridge network named `protypilot`. All three services attach to it; the frontend resolves `backend` to the backend container's IP by service name.
- One named volume, `protypilot_pgdata`, for Postgres storage.

6. Local Verification

6.1 Prereqs

- Docker Desktop running.
- A `.env` at the repository root with at least `JWT_SECRET=<16+ chars>` set. The other values (`DATABASE_URL`, `FRONTEND_URL`, `PORT`) are overridden by compose so they do not need to be set for the Docker workflow.

6.2 Bring up the full stack

```
docker compose up -d --build
```

First build is ~60–90 seconds (mostly `npm ci`). Subsequent builds are seconds — only the source-copy layers are re-run.

6.3 Confirm health

```
docker compose ps
```

You want all three rows in the `(healthy)` state. The backend usually takes ~10 seconds to flip from `(health: starting)` to `(healthy)` because its first action is to run migrations.

```
curl http://localhost:4000/api/health      # Backend direct -> {"status":"ok"}
curl http://localhost:8080/                # Frontend bundle -> HTML
curl http://localhost:8080/api/health      # Proxied via nginx -> {"status":"ok"}
curl http://localhost:8080/api/properties  # No JWT -> 401
```

6.4 Open the app

Visit <http://localhost:8080> in a browser, register an account, and verify the dashboard loads.

6.5 Bring the stack down

```
docker compose down
```

The Postgres volume persists. To reset everything including data:

```
docker compose down -v
```

7. Render Deployment

The production target is **Render** (free tier). Mapping each compose service to a Render component:

Compose service	Render component	Notes
postgres	PostgreSQL (managed)	Render-issued <code>DATABASE_URL</code> . Run <code>npm run prisma:migrate -w backend</code> ONCE locally against that URL, or rely on the container's startup migration (preferred).
backend	Web Service (Docker)	Set the Dockerfile path to <code>backend/Dockerfile</code> and the build context to the repository root. Set <code>JWT_SECRET</code> , <code>DATABASE_URL</code> (from the Render Postgres service), and <code>FRONTEND_URL</code> (the deployed frontend's URL) as environment variables in the Render dashboard.
frontend	Static Site	Build command: <code>npm install && npm run build -w frontend</code> . Publish directory: <code>frontend/dist</code> . The nginx-based Docker image is not used in production because Render serves static sites with its own CDN. The reverse-proxy that nginx provides locally is instead handled by configuring a "Rewrite" rule on the Render Static Site: <code>/api/*</code> → <code>https://<backend-service>.onrender.com/api/*</code> .

CORS: when running on Render, set the backend's `FRONTEND_URL` env var to the static site's URL (e.g. `https://propertyilot.onrender.com`). The exact URL is assigned once the static site is created.

Full provider rationale (Render vs. alternatives) is captured in [cloud-provider-justification.md](#) (added under Task 4.A.1).

8. Troubleshooting

Backend container restarts in a loop with "Prisma schema validation - The datasource property `url` is no longer supported"

The container is downloading Prisma 7 from the registry at startup instead of using the locally installed Prisma 6 CLI. This happened during development when the `CMD` used `npm run prisma` and the bin lookup failed. The fix already in `backend/Dockerfile` is to invoke the bin by its concrete path: `./node_modules/.bin/prisma migrate deploy`. If you see this again, confirm that:

- The runtime stage copies **both** `/repo/node_modules` **and** `/repo/backend/node_modules` from the builder.
- The container `WORKDIR` is `/app/backend` so the relative `./node_modules/.bin/prisma` lookup resolves to the workspace-local bin.

Backend container restarts with "JWT_SECRET must be set..."

The compose file's `${JWT_SECRET:?...}` interpolation is intentionally strict. Add `JWT_SECRET=<long-random-string>` to your root `.env` and re-run `docker compose up`.

502 Bad Gateway from `/api/*` via the frontend

nginx is up but it can't reach the backend. Either the backend is still starting (check `docker compose ps`) or the backend service name resolution failed because the two services aren't on the same network. The compose file puts both on the `propertyilot` bridge — verify with `docker network inspect propertyilot` and confirm both containers are listed.

Healthcheck reports `unhealthy` even though the app works

Earlier versions of the compose file used `http://localhost/...` in the healthcheck. Alpine's busybox `wget` sometimes fails to resolve `localhost` from a shell invocation. The fix in place is to use `http://127.0.0.1/...`. If you change these, keep them on the IP literal.

Stale image: changes don't take effect

Docker layer caching can hold an old `npm ci` layer across config changes. Force a clean rebuild:

```
docker compose build --no-cache backend
docker compose up -d backend
```

For a full reset:

```
docker compose down -v
docker compose up -d --build
```

propertypilot_pgdata volume corrupted from an earlier dev DB

```
docker compose down -v
docker volume rm propertypilot_pgdata # belt-and-suspenders
docker compose up -d --build
```

The startup migration will recreate the schema and the seed user (via `npm run db:seed -w backend` if you want it — that step is **not** run automatically by the Docker stack).

9. Maintenance Tasks Specific to Docker

Task	Command
Tail the backend logs	<code>docker compose logs -f backend</code>
Shell into the backend container	<code>docker compose exec backend sh</code>
Rebuild only the frontend image	<code>docker compose build frontend && docker compose up -d frontend</code>
Pull the latest base images (security patches)	<code>docker compose pull then docker compose up -d --build</code>
Inspect the bridge network	<code>docker network inspect propertypilot</code>
Back up the database	<code>docker compose exec postgres pg_dump -U propertypilot propertypilot > backup.sql</code>

For non-Docker maintenance (local dev, migrations from the host), see [maintenance-guide.md](#) .